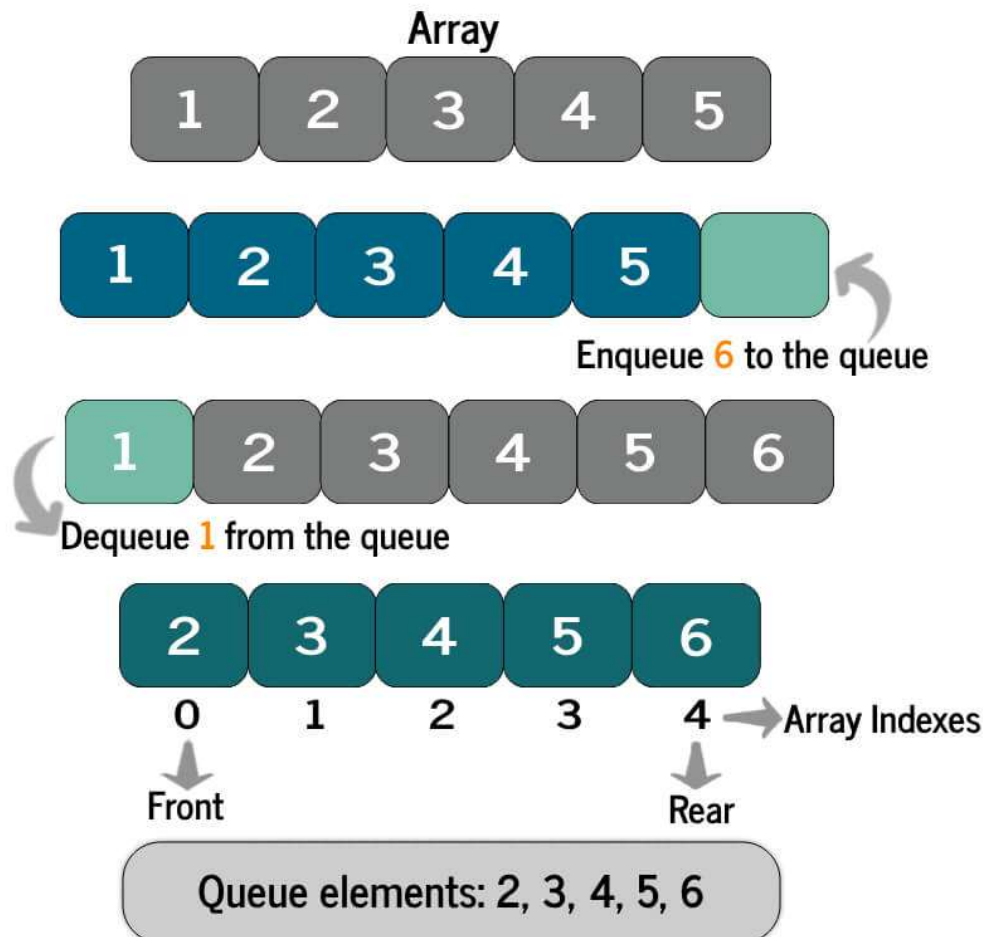


Queue using Array

A queue is a linear data structure that operates on the principle of First In, First Out (FIFO). It is widely used in various computing problems where tasks or resources are processed in the same order they are added. The queue can be implemented in different ways, and one of the most efficient ways is through an array.

Array Implementation of Queue





What is Queue using Array?

Queues are linear data structures that follow the First In First Out (FIFO) principle. As a result, the element that is inserted first will be removed first. A queue has two primary operations:

- **Enqueue:** Insert an element into the queue (at the rear).
- **Dequeue:** Remove an element from the queue (from the front).

The queue is typically represented by two variables, they are:

- **Front:** Refers to the queue's first element.
- **Rear:** Refers to the last element of the queue.

Array Representation of Queue

In an array-based implementation:

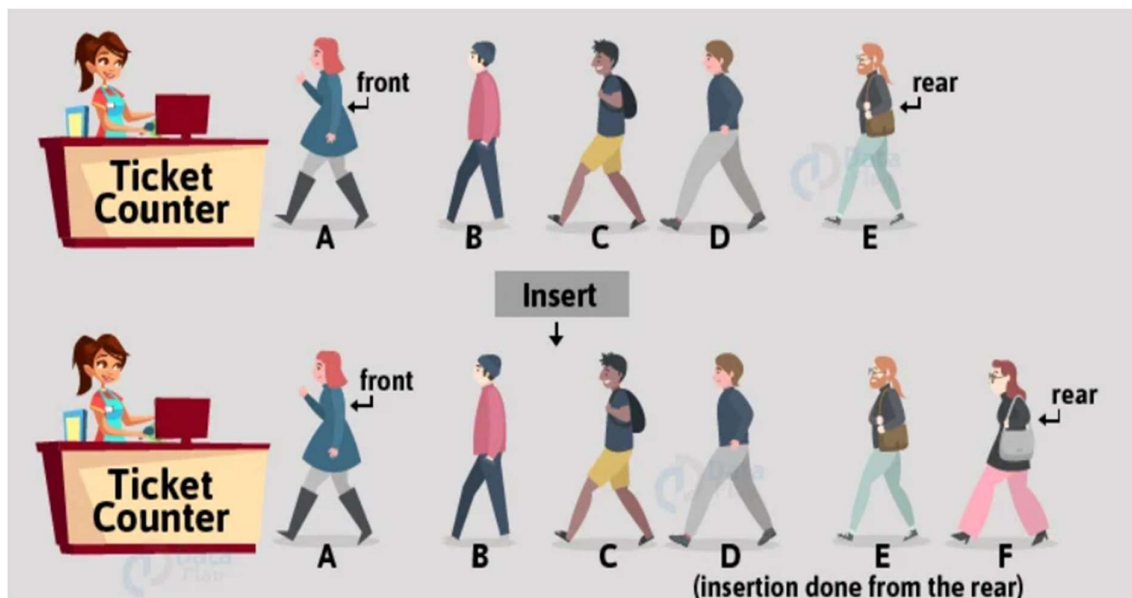
- The queue will be represented by an array of a fixed size.
- The front pointer points to the index of the front of the queue, and the rear pointer points to the index of the rear of the queue.

Basic Operations of Queue Using Array

The basic operations of a queue using an array are:

Enqueue Operation (Insertion)

- Insert a new element at the position indicated by the rear pointer.
- Increment the rear pointer after the insertion.
- If the queue is full (when rear exceeds the size of the array), the insertion is not possible.

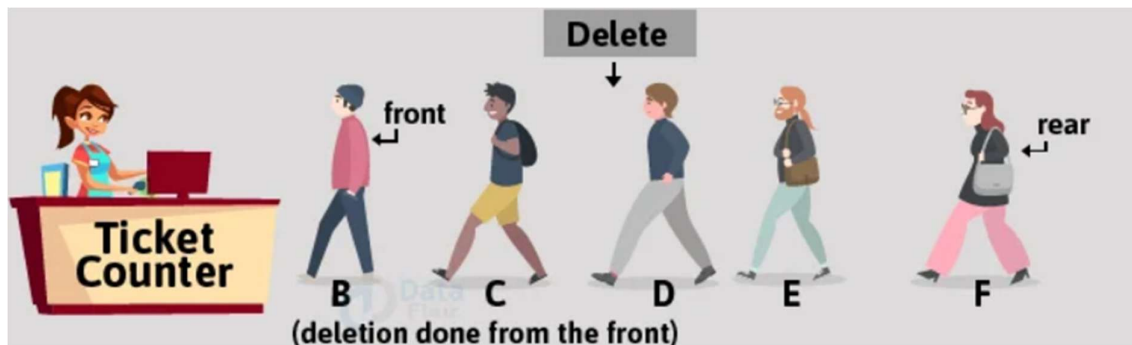


Algorithm for Enqueue Operation

1. Check if the queue is full ($\text{rear} == \text{size of the array}$).
2. If the queue is full, print "Queue is full" and exit.
3. If the queue is not full:
 - a. Increment the rear index.
 - b. Insert the new element at the rear index in the array.

Deque Operation (Deletion)

- Remove the element at the position indicated by the front pointer.
- Increment the front pointer after the deletion.
- If the queue is empty (front equals rear), the deletion is not possible.



Algorithm for Dequeue Operation

1. Check if the queue is empty ($\text{front} == \text{rear}$).
2. If the queue is empty, print "Queue is empty" and exit.
3. If the queue is not empty:
 - a. Get the value at the front index of the array.
 - b. Increment the front index.

Example of Queue Using Array

Let's use an array of size 5 to demonstrate the queue operations with an example.

Initial Setup

We start with two variables, front and rear both initialized to -1. This indicates that the queue is empty initially.

- Array: []
- front = -1 (indicates the queue is empty)
- rear = -1 (indicates the queue is empty)

Front = (-1) 
 Rear = (-1) 0 1 2 3
 (empty queue)

Operation 1: Enqueue(10)

Before Operation: front = -1, rear = -1

We insert the value 10 at the rear of the queue. Since the queue is empty, both front and rear will point to index 0.

After Operation:

- front = 0, rear = 0
- Queue structure: [10]

Front = 0 
 Rear = 0 0 1 2 3

Operation 2: Enqueue(20)

Before Operation: front = 0, rear = 0

We insert the value 20 at the rear of the queue. The rear is incremented to 1, and the value 20 is added at position 1.

After Operation:

- front = 0, rear = 1
- Queue structure: [10, 20]

Front = 0
Rear = 1

10	20		
0	1	2	3

Operation 3: Dequeue()

Before Operation: front = 0, rear = 1

We remove the value at the front of the queue, which is 10 (the value at index 0). After removing this element, the front is incremented to 1.

After Operation:

- front = 1, rear = 1
- Queue structure: [20]

Front = 1
Rear = 1

	20		
0	1	2	3

Operation 4: Enqueue(30)

Before Operation: front = 1, rear = 1

We insert the value 30 at the rear of the queue. The rear is incremented to 2, and the value 30 is added at position 2.

After Operation:

- front = 1, rear = 2
- Queue structure: [20, 30]

Front = 1
Rear = 2

	20	30	
0	1	2	3

Operation 5: Enqueue(40)

Before Operation: front = 1, rear = 2

We insert the value 40 at the rear of the queue. The rear is incremented to 3, and the value 40 is added at position 3.

After Operation:

- front = 1, rear = 3
- Queue structure: [20, 30, 40]

Front = 1

	20	30	40
--	----	----	----

Rear = 3 0 1 2 3

Operation 6: Dequeue()

Before Operation: front = 1, rear = 3

We remove the value at the front of the queue, which is 20 (the value at index 1). After removing this element, the front is incremented to 2.

After Operation:

- front = 2, rear = 3
- Queue structure: [30, 40]

Front = 2

		30	40
--	--	----	----

Rear = 3 0 1 2 3

State of the Queue After Operations

Operation	Front	Rear	Queue Array
-----------	-------	------	-------------

Initial	-1	-1	[]
---------	----	----	-----

Enqueue(10)	0	0	[10]
-------------	---	---	------

Enqueue(20)	0	1	[10,20]
-------------	---	---	---------

Operation	Front	Rear	Queue Array
-----------	-------	------	-------------

Dequeue	1	1	[20]
---------	---	---	------

Enqueue(30)	1	2	[20,30]
-------------	---	---	---------

Enqueue(40)	1	3	[20,30,40]
-------------	---	---	------------

Dequeue()	2	3	[30,40]
-----------	---	---	---------

Why Use an Array to Implement a Queue?

Using an array to implement a queue has several advantages. Arrays are contiguous blocks of memory, which allows for easy access to elements via an index. This makes it easy to manage the front and rear of the queue efficiently. The main reasons to use an array for a queue implementation are:

- An array has a predefined size, which helps in limiting the number of elements in the queue.
- Since arrays provide constant-time access to elements ($O(1)$ complexity), enqueue and dequeue operations can be performed with minimal overhead.
- Implementing a queue using an array is simple, making it an ideal choice for basic queue operations.

Implementation of Queue using array

```
#include<iostream>
```

```
using namespace std;
```

```
int q[20],rear=-1,front=-1,m=5;
```



```

int insert(int elem)
{
    if(rear<=m-1)
    {
        if(rear==-1 || front==-1)
        {
            q[++rear]=elem;
            front++;
        }
        else
            q[++rear]=elem;
        cout<<"inserted element = "<<q[rear]<<endl;
    }
    else
    {
        cout<<"queue is full"<<endl;
    }
    cout<<"front= "<<front<<endl;
    cout<<"rear="<<rear<<endl;
}

```

```

void dequeue()
{
    if(front==-1 || front>rear)
    {
        cout<<endl;
        cout<<"Queue is empty"<<endl;
    }
    else
    {
        cout<<"deleted element = "<<q[front]<<endl;
        front++;
    }
}

void display()
{
    if (front == -1 || front > rear)
        cout << "Queue is empty" << endl;
    else {
        cout << "Queue elements are: ";
        for(int i=front;i<=rear;i++)
            cout<<" "<<q[i];
    }
}

```

```
}  
  
cout <<endl;  
  
}  
  
int main()  
{  
  
    insert(5);  
  
    insert(6);  
  
    insert(7);  
  
    display();  
  
    dequeue();  
  
        display();  
  
    insert(1);  
  
    insert(9);  
  
    insert(3);  
  
    display();  
  
    dequeue();  
  
    display();  
  
    dequeue();  
  
    display();  
  
    dequeue();  
  
    display();  
  
}
```

```
    dequeue();  
    display();  
    dequeue();  
    display();  
    dequeue();  
}
```

Reference:

<https://www.ccbp.in/blog/articles/implement-queue-using-array>

<https://www.educba.com/array-implementation-of-queue>

<https://data-flair.training/blogs/queue-in-c-cpp>

